

Exercise 1: Control Structures

Scenario 1: Apply 1% Discount to Customers Above 60 Years

```
DECLARE
    v_age NUMBER;
BEGIN
    FOR rec IN (
        SELECT c.CustomerID, l.LoanID, l.InterestRate, c.DOB
        FROM Customers c
        JOIN Loans l ON c.CustomerID = l.CustomerID
    ) LOOP

        v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, rec.DOB) / 12);

        IF v_age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE LoanID = rec.LoanID;
        END IF;

    END LOOP;

    COMMIT;
END;
/
```

Scenario 2: Set VIP Status for Customers with Balance > 10000

```
ALTER TABLE Customers ADD IsVIP VARCHAR2(5);

BEGIN
    FOR rec IN (SELECT CustomerID, Balance FROM Customers) LOOP

        IF rec.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = rec.CustomerID;
        ELSE
            UPDATE Customers
            SET IsVIP = 'FALSE'
            WHERE CustomerID = rec.CustomerID;
        END IF;

    END LOOP;
```

```
    COMMIT;
END;
/
```

Scenario 3: Send Loan Due Reminders

```
BEGIN
  FOR rec IN (
    SELECT c.Name, l.LoanID, l.EndDate
    FROM Customers c
    JOIN Loans l
    ON c.CustomerID = l.CustomerID
    WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30
  ) LOOP

    DBMS_OUTPUT.PUT_LINE(
      'Reminder: Loan ' || rec.LoanID ||
      ' for customer ' || rec.Name ||
      ' is due on ' || rec.EndDate
    );

  END LOOP;
END;
/
```

Exercise 2: Error Handling

Scenario 1: SafeTransferFunds Procedure

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
  p_from_account NUMBER,
  p_to_account NUMBER,
  p_amount NUMBER
)
IS
  v_balance NUMBER;
BEGIN

  SELECT Balance
  INTO v_balance
  FROM Accounts
  WHERE AccountID = p_from_account;

  IF v_balance < p_amount THEN
    RAISE_APPLICATION_ERROR(
```

```

        -20001,
        'Insufficient Funds'
    );
END IF;

UPDATE Accounts
SET Balance = Balance - p_amount
WHERE AccountID = p_from_account;

UPDATE Accounts
SET Balance = Balance + p_amount
WHERE AccountID = p_to_account;

COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE(
            'Transfer Failed: ' || SQLERRM
        );
END;
/

```

Scenario 2: UpdateSalary Procedure

```

CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_empid NUMBER,
    p_percent NUMBER
)
IS
BEGIN

    UPDATE Employees
    SET Salary = Salary + (Salary * p_percent / 100)
    WHERE EmployeeID = p_empid;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE(

```

```

        'Employee ID Not Found'
    );

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/

```

Scenario 3: AddNewCustomer Procedure

```

CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_id NUMBER,
    p_name VARCHAR2,
    p_dob DATE,
    p_balance NUMBER
)
IS
BEGIN

    INSERT INTO Customers
    (
        CustomerID,
        Name,
        DOB,
        Balance,
        LastModified
    )
    VALUES
    (
        p_id,
        p_name,
        p_dob,
        p_balance,
        SYSDATE
    );

    COMMIT;

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE(
            'Customer ID Already Exists'
        );

```

```
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE(
                'Error: ' || SQLERRM
            );
        END;
    /
```

Exercise 3: Stored Procedures

Scenario 1: ProcessMonthlyInterest

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;
END;
/
```

Scenario 2: UpdateEmployeeBonus

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department VARCHAR2,
    p_bonus_percent NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary +
        (Salary * p_bonus_percent / 100)
    WHERE Department = p_department;

    COMMIT;
END;
/
```

Scenario 3: TransferFunds

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_account NUMBER,
    p_to_account NUMBER,
```

```

    p_amount NUMBER
)
IS
    v_balance NUMBER;
BEGIN

    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Insufficient Balance'
        );
    END IF;

    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_account;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_account;

    COMMIT;
END;
/

```

Exercise 4: Functions

Scenario 1: CalculateAge

```

CREATE OR REPLACE FUNCTION CalculateAge(
    p_dob DATE
)
RETURN NUMBER
IS
    v_age NUMBER;
BEGIN
    v_age :=
        FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);

    RETURN v_age;

```

```
END;  
/
```

Example

```
SELECT CalculateAge(DOB)  
FROM Customers;
```

Scenario 2: CalculateMonthlyInstallment

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(  
    p_loan_amount NUMBER,  
    p_interest_rate NUMBER,  
    p_years NUMBER  
)  
RETURN NUMBER  
IS  
    v_monthly NUMBER;  
BEGIN  
    v_monthly :=  
        (p_loan_amount +  
        (p_loan_amount * p_interest_rate / 100))  
        / (p_years * 12);  
  
    RETURN ROUND(v_monthly, 2);  
END;  
/
```

Example

```
SELECT CalculateMonthlyInstallment(  
    50000,  
    5,  
    5)  
FROM dual;
```

Scenario 3: HasSufficientBalance

```
CREATE OR REPLACE FUNCTION HasSufficientBalance(  
    p_account_id NUMBER,  
    p_amount NUMBER  
)  
RETURN VARCHAR2  
IS  
    v_balance NUMBER;  
BEGIN
```

```

SELECT Balance
INTO v_balance
FROM Accounts
WHERE AccountID = p_account_id;

IF v_balance >= p_amount THEN
    RETURN 'TRUE';
ELSE
    RETURN 'FALSE';
END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'FALSE';
END;
/

```

Example

```

SELECT HasSufficientBalance(
    1,
    500)
FROM dual;

```

Exercise 5: Triggers

Scenario 1: UpdateCustomerLastModified

```

CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/

```

Scenario 2: LogTransaction

Audit Table

```

CREATE TABLE AuditLog (
    AuditID NUMBER,
    TransactionID NUMBER,

```



```

    AccountID NUMBER,
    ActionDate DATE
);

Trigger

CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog
    VALUES(
        AuditLog_seq.NEXTVAL,
        :NEW.TransactionID,
        :NEW.AccountID,
        SYSDATE
    );
END;
/

```

Scenario 3: CheckTransactionRules

```

CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
    v_balance NUMBER;
BEGIN

    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = :NEW.AccountID;

    IF :NEW.TransactionType = 'Withdrawal' THEN

        IF :NEW.Amount > v_balance THEN
            RAISE_APPLICATION_ERROR(
                -20001,
                'Insufficient Balance'
            );
        END IF;

    ELSIF :NEW.TransactionType = 'Deposit' THEN

        IF :NEW.Amount <= 0 THEN
            RAISE_APPLICATION_ERROR(

```

```

        -20002,
        'Deposit Must Be Positive'
    );
END IF;

END IF;
END;
/

```

Exercise 6: Cursors

Scenario 1: GenerateMonthlyStatements

```

DECLARE
    CURSOR GenerateMonthlyStatements IS
    SELECT *
    FROM Transactions
    WHERE TO_CHAR(TransactionDate,'MMYYYY')
        = TO_CHAR(SYSDATE,'MMYYYY');

    v_rec Transactions%ROWTYPE;
BEGIN

    OPEN GenerateMonthlyStatements;

    LOOP
        FETCH GenerateMonthlyStatements
        INTO v_rec;

        EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Transaction ID: ' ||
            v_rec.TransactionID ||
            ' Amount: ' ||
            v_rec.Amount
        );
    END LOOP;

    CLOSE GenerateMonthlyStatements;
END;
/

```

Scenario 2: ApplyAnnualFee

```
DECLARE
  CURSOR ApplyAnnualFee IS
    SELECT AccountID
    FROM Accounts;

BEGIN

  FOR rec IN ApplyAnnualFee LOOP

    UPDATE Accounts
    SET Balance = Balance - 100
    WHERE AccountID = rec.AccountID;

  END LOOP;

  COMMIT;
END;
/
```

Scenario 3: UpdateLoanInterestRates

```
DECLARE
  CURSOR UpdateLoanInterestRates IS
    SELECT LoanID, InterestRate
    FROM Loans;

BEGIN

  FOR rec IN UpdateLoanInterestRates LOOP

    UPDATE Loans
    SET InterestRate = rec.InterestRate + 0.5
    WHERE LoanID = rec.LoanID;

  END LOOP;

  COMMIT;
END;
/
```

Exercise 7: Packages

Scenario 1: CustomerManagement Package

Package Specification

CREATE OR REPLACE PACKAGE CustomerManagement AS

```
    PROCEDURE AddCustomer(  
        p_id NUMBER,  
        p_name VARCHAR2,  
        p_dob DATE,  
        p_balance NUMBER  
    );
```

```
    PROCEDURE UpdateCustomer(  
        p_id NUMBER,  
        p_name VARCHAR2  
    );
```

```
    FUNCTION GetCustomerBalance(  
        p_id NUMBER  
    ) RETURN NUMBER;
```

```
END CustomerManagement;  
/
```

Package Body

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

```
    PROCEDURE AddCustomer(  
        p_id NUMBER,  
        p_name VARCHAR2,  
        p_dob DATE,  
        p_balance NUMBER  
    )  
    IS  
    BEGIN  
        INSERT INTO Customers  
        VALUES(  
            p_id,  
            p_name,  
            p_dob,  
            p_balance,  
            SYSDATE  
        );  
    END;
```

```

PROCEDURE UpdateCustomer(
    p_id NUMBER,
    p_name VARCHAR2
)
IS
BEGIN
    UPDATE Customers
    SET Name = p_name
    WHERE CustomerID = p_id;
END;

FUNCTION GetCustomerBalance(
    p_id NUMBER
)
RETURN NUMBER
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Customers
    WHERE CustomerID = p_id;

    RETURN v_balance;
END;

END CustomerManagement;
/

```

Scenario 2: EmployeeManagement Package

Package Specification

CREATE OR REPLACE PACKAGE EmployeeManagement AS

```

PROCEDURE HireEmployee(
    p_id NUMBER,
    p_name VARCHAR2,
    p_position VARCHAR2,
    p_salary NUMBER,
    p_department VARCHAR2
);

PROCEDURE UpdateEmployee(
    p_id NUMBER,

```

```
    p_salary NUMBER  
);
```

```
FUNCTION AnnualSalary(  
    p_id NUMBER  
) RETURN NUMBER;
```

```
END EmployeeManagement;  
/
```

Package Body

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

```
PROCEDURE HireEmployee(  
    p_id NUMBER,  
    p_name VARCHAR2,  
    p_position VARCHAR2,  
    p_salary NUMBER,  
    p_department VARCHAR2  
)  
IS  
BEGIN  
    INSERT INTO Employees  
    VALUES(  
        p_id,  
        p_name,  
        p_position,  
        p_salary,  
        p_department,  
        SYSDATE  
    );  
END;
```

```
PROCEDURE UpdateEmployee(  
    p_id NUMBER,  
    p_salary NUMBER  
)  
IS  
BEGIN  
    UPDATE Employees  
    SET Salary = p_salary  
    WHERE EmployeeID = p_id;  
END;
```

```
FUNCTION AnnualSalary(  
    p_id NUMBER
```

```

)
RETURN NUMBER
IS
    v_salary NUMBER;
BEGIN
    SELECT Salary
    INTO v_salary
    FROM Employees
    WHERE EmployeeID = p_id;

    RETURN v_salary * 12;
END;

END EmployeeManagement;
/

```

Scenario 3: AccountOperations Package

Package Specification

```
CREATE OR REPLACE PACKAGE AccountOperations AS
```

```

    PROCEDURE OpenAccount(
        p_accountid NUMBER,
        p_customerid NUMBER,
        p_type VARCHAR2,
        p_balance NUMBER
    );

```

```

    PROCEDURE CloseAccount(
        p_accountid NUMBER
    );

```

```

    FUNCTION GetTotalBalance(
        p_customerid NUMBER
    ) RETURN NUMBER;

```

```

END AccountOperations;
/

```

Package Body

```
CREATE OR REPLACE PACKAGE BODY AccountOperations AS
```

```

    PROCEDURE OpenAccount(
        p_accountid NUMBER,
        p_customerid NUMBER,

```

```

        p_type VARCHAR2,
        p_balance NUMBER
    )
    IS
    BEGIN
        INSERT INTO Accounts
        VALUES(
            p_accountid,
            p_customerid,
            p_type,
            p_balance,
            SYSDATE
        );
    END;

PROCEDURE CloseAccount(
    p_accountid NUMBER
)
    IS
    BEGIN
        DELETE FROM Accounts
        WHERE AccountID = p_accountid;
    END;

FUNCTION GetTotalBalance(
    p_customerid NUMBER
)
    RETURN NUMBER
    IS
        v_total NUMBER;
    BEGIN
        SELECT SUM(Balance)
        INTO v_total
        FROM Accounts
        WHERE CustomerID = p_customerid;

        RETURN NVL(v_total,0);
    END;

END AccountOperations;
/

```